# longops

Long operations and detailed calculations.
Opérations posées et calculs détaillés.

v0.2.2 Akilon <https://github.com/Akilon27> MIT

## About

As a mathematics teacher, I sometimes need to show how to carry out arithmetic “by hand”. I am now making available the functions that I originally created for my own use.

I previously used the LaTeX package **xlop**, and since Typst is much more convenient to use, I decided to create my own equivalent. This package is **not** a direct translation of **xlop**: it includes some features that **xlop** does not, while omitting others, according to my own needs.

I occasionally used AI tools (mainly Gemini for the **Order of Operations** module, and also to add features to the other functions that I had initially implemented for just one of them).

**Acknowledgments** : I wanted to thank ObaulG, the part “durations” is mostly is work.

## Functions for handwritten arithmetic layouts ...


- from-base()
- to-base()
- base-to-base()
- addition()
- soustraction()
- multiplication()
- egalite-euclidienne()
- egalite-decimale()
- division()
- racine()
- add-en-ligne()
- sous-en-ligne()
- grid-multiplication()
- binary-multiplication()
- lines-multiplication()
- area-multiplication()


### from-base

conversion to decimal from another base (defaut : 2).

```
#from-base("1AF",base:16, fr: false)
```

```
431
```

### Parameters

```
from-base(
  val: int float str,
  base: int,
  fr: boolean,
  show-base: boolean
) -> str int float
```

**val** `int` or `float` or `str`

number to convert

**base** `int`

origin base

Default: `2`

**fr** `boolean`

mode fr to get "," instead of "." as a decimal separator

Default: `false`

**show-base** `boolean`

show the base of the result

Default: `false`

**to-base**
conversion from decimal to another base (defaut : 2).

```
#to-base(base:16,"7.8",fr: false)
```

```
7.CCCCCC
```

```
#to-base(100, base: 8, fr: false)
```

```
144
```

**Parameters**

```
to-base(
  val: int float str,
  base: int,
  fr: boolean,
  show-base: boolean,
  precision: int
) -> str int float
```

**val** `int` or `float` or `str`

number to convert

**base** `int`

base to convert to

Default: `2`

**fr** `boolean`

mode fr to get “,” instead of “.” as a decimal separator

Default: `false`

**show-base** `boolean`

show the base of the result

Default: `false`

**precision** `int`

to limit the number of digits if the fractional part is infinite

Default: 6

**base-to-base**

conversion from a base (defaut : 2) to another (defaut : 10).

```
// Conversion between bases (16 to 2):
#base-to-base("1A", base-from: 16, base-to: 2, equal: true, fr: false)
```

$$(1\mathrm{A})_{16} = (11010)_{2}$$

```
#base-to-base("77", base-from: 8, base-to: 2, equal: true, mode:"line", fr: false)
```

$$\overline{77}^{8} = \overline{111111}^{2}$$

```
// Full explanation (Base 16 to Base 8):
#base-to-base("1A.F", base-from: 16, base-to: 8, explain: true,mode: "line", fr: false)
```

**1. Conversion from base 16 to base 10:**

$$\overline{1\mathrm{A.F}}^{16} = 1 \times 16^{1} + 10 \times 16^{0} + 15 \times 16^{-1} = \overline{26.9375}^{10}$$

**2. Conversion from base 10 to base 8:**

*Integer part (successive divisions by 8) :*

- $26 = 8 \times 3 + \mathbf{2}$
- $3 = 8 \times 0 + \mathbf{3}$

*Fractional part (successive multiplications by 8) :*

- $0.9375 \times 8 = \mathbf{7.5}$ (integer part: **7** )
- $0.5 \times 8 = \mathbf{4}$ (integer part: **4** )

---

**Result:** $\overline{1\mathrm{A.F}}^{16} = \overline{32.74}^{8}$

**Parameters**

```
base-to-base(
  val: int float str,
  base-from: int,
  base-to: int,
  explain: boolean,
  equal: boolean,
  precision: int,
  fr: boolean,
  mode: str
) -> str int float
```

**val** `int` or `float` or `str`

number to convert

**base-from** `int`

original base

Default: `2`

**base-to** `int`

end base

Default: `10`

**explain** `boolean`

to get explanation of the conversion

Default: `false`

**equal** `boolean`

to get the conversion equality

Default: `false`

**precision** `int`

to limit the number of digits if the fractional part is infinite

Default: `6`

**fr** `boolean`

mode fr to get “,” instead of “.”

Default: `false`

**mode** `str`

a few modes to show the bases “paren”, “line”, “simple”

Default: `"paren"`

**addition**

**Parameters**

```
addition(
  base: int,
  show-base: boolean,
  show-carry: boolean,
  carries-color: color,
  size: length relative length,
  fr: boolean,
  hide-result: boolean,
  border-color: color,
  solution-color: color,
  type-mask: str,
  list: array,
  solution: boolean,
  sign: boolean content,
  space: length relative length fraction,
  text-size: length relative length,
  italian-style: boolean,
  ..args: array arguments
) -> content
```

**base** `int`

Base (from 2 to 36)

Default: `10`

**show-base** `boolean`

to show the base of the numbers and operation

Default: `false`

**show-carry** `boolean`

show or hide the carries

Default: `true`

**carries-color** `color`

color of the carries

Default: `red`

**size** `length` or `relative length`

Size of the columns

Default: `1em`

**fr** `boolean`

french decimal separaro “,” or not

Default: `true`

**hide-result** `boolean`

Hide the carry row and the result

Default: `false`

**border-color** `color`

Color of the mask of hidden digits

Default: `blue.mix(gray)`

**solution-color** `color`

Color of the hidden digits if solution = true

Default: `red`

**type-mask** `str`

How the digits are hidden : “rect” for a round rectangle or “” for dots

Default: `"rect"`

**list** `array`

list of the digits to hide

Default: `()`

**solution** `boolean`

show the carries and the hidden digits

Default: `false`

**sign** `boolean` or `content`

Sign of equality, false or a symbol $ = $

Default: `false`

**space** `length` or `relative length` or `fraction`

Space if more than one calculation on a line, default : 1fr

Default: `1fr`

**text-size** `length` or `relative length`

size of the text : 1em

Default: `1em`

**italian-style** `boolean`

Italian style operation with the operand on the right

Default: `false`

**..args** `array` or `arguments`

Numbers to add

**soustraction**

**Parameters**

```
soustraction(
  base: int,
  show-base: boolean,
  show-borrow: boolean,
  canceled: boolean,
  fr: boolean,
  carries-color: color,
  size: length relative length,
  zeros: boolean,
  hide-result: boolean,
  border-color: color,
  solution-color: color,
  type-mask: str,
  list: array,
  solution: boolean,
  sign: boolean content,
  text-size: length relative length,
  space: length relative length fraction,
  italian-style: boolean,
  ..args: array arguments
) -> content
```

**base** `int`

Base (from 2 to 36)

Default: `10`

**show-base** `boolean`

to show the base of the numbers and operation

Default: `false`

**show-borrow** `boolean`

Show or hide the borrows

Default: `true`

**canceled** `boolean`

If fr:false, show or hide the crossed-out original digit

Default: `false`

**fr** `boolean`

French notation or English borrowing method

Default: `true`

**carries-color** `color`

Color of the borrows

Default: `red`

**size** `length` or `relative length`

Column width

Default: `1em`

**zeros** `boolean`

Add trailing zeros when needed

Default: `true`

**hide-result** `boolean`

Hide the borrows and the result

Default: `false`

**border-color** `color`

Color of the border around hidden digits

Default: `blue.mix(gray)`

**solution-color** `color`

Color of hidden digits when solution = true

Default: `red`

**type-mask** `str`

How hidden digits are displayed: “rect” for a rounded rectangle or …

Default: `"rect"`

**list** `array`

List of digits to hide in a fill-in-the-blank subtraction

Default: `()`

**solution** `boolean`

Show the borrows and the result in hide-result mode (for answer keys)

Default: `false`

**sign** `boolean` or `content`

Equality sign, false or a symbol $ = $

Default: `false`

**text-size** `length` or `relative length`

Text size: 1em

Default: `1em`

**space** `length` or `relative length` or `fraction`

Spacing when several calculations are on the same line

Default: `1fr`

**italian-style** `boolean`

Italian style operation with the operand on the right

Default: `false`

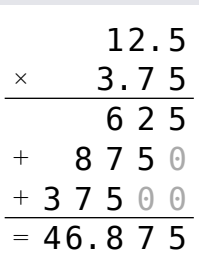
**..args** `array` or `arguments`

Numbers

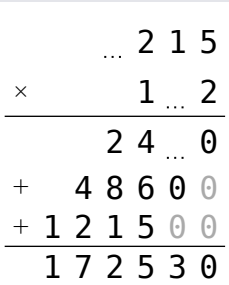
**multiplication**
Lay out the multiplication of 2 integer or decimal numbers with the possibility of hiding intermediate rows using `hide-result`, hiding a `list` of selected digits, then displaying the `solution`, with various configurable colors.

**Examples:**

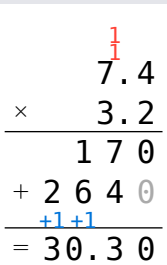
```
#multiplication(12.5, 3.75, fr: false)
```



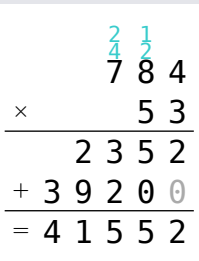

```
#multiplication(1215, 142,list:
(3,12,19,36),type-mask: "line", fr:
false)
```



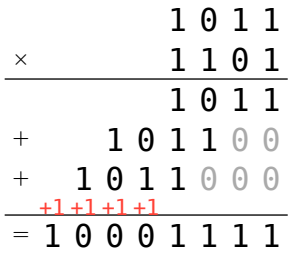

```
// In base 8 with all the carries and
floating numbers:
#multiplication(base: 8, "7.4",
"3.2",carries: true,carries-color:
(red,blue), fr: false)
```




```
// with only the multiplication
carries:
#multiplication(carries: "mult", 784,
53, fr: false, carries-color: teal)
```




```
// in binary with only the addition
carries:
#multiplication(base: 2, carries:
"add", "1011", "1101", fr: false)
```



### Parameters

```
multiplication(
  a: int float str,
  b: int float str,
  base: int,
  show-base: boolean,
  carries: boolean str,
  carries-color: color array,
  mode: str,
  sym: boolean str,
  fr: boolean str,
  size: length relative length,
  hide-result: boolean,
  list: array,
  solution: boolean,
  type-mask: str,
  border-color: color,
  solution-color: color,
  italian-style: boolean,
  zero-lines: boolean,
  space: length relative length fraction
) -> content
```

**a** `int` or `float` or `str`

first factor

**b** `int` or `float` or `str`

second factor

**base** `int`

Base (2 to 36)

Default: `10`

**show-base** `boolean`

to show in which base the numbers and operation are

Default: `false`

**carries** `boolean` or `str`

to show the carries : false, true (“all”), “mult”, “add”

Default: `false`

**carries-color** `color` or `array`

Color of the carries

Default: `red`

**mode** `str`

Display mode for shifted rows: “g” (default) for gray zeros, “0” for normal zeros, “p” for small dots, “gp” for larger dots, “” or anything else for nothing

Default: `"g"`

**sym** `boolean` or `str`

Show the addition “+” signs and the final “=” if true; only “+” if “+”, only “=” if “=”, otherwise nothing

Default: `true`

**fr** `boolean` or `str`

If fr:true, the decimal separator is “,” otherwise “.”

Default: `true`

**size** `length` or `relative length`

Column width

Default: `1em`

**hide-result** `boolean`

Hide intermediate rows and the final result for exercises

Default: `false`

**list** `array`

List of cell indices to hide (fill-in-the-blank multiplication)

Default: `()`

**solution** `boolean`

Show or hide the solution inside the masks

Default: `false`

**type-mask** `str`

Mask type (“rect” or “line”)

Default: `"rect"`

**border-color** `color`

Color of the mask border

Default: `blue.mix(gray)`

**solution-color** `color`

Color of the solution text

Default: `red`

**italian-style** `boolean`

Italian style operation with the operand on the right

Default: `false`

**zero-lines** `boolean`

To show the line(s) of product by a factor of 0

Default: `false`

**space** `length` or `relative length` or `fraction`

Spacing when several calculations are on the same line

Default: `1fr`

### egalite-euclidienne

Write the equality corresponding to a Euclidean division.

```
#egalite-euclidienne(126, 27)
```

$$126 = 4 \times 27 + 18$$

**Parameters**

```
egalite-euclidienne(
  a: int float,
  b: int float
) -> content
```

**a** `int` or `float`

dividend

**b** `int` or `float`

divisor

### egalite-decimale

Write the equality corresponding to a decimal division.

```
#egalite-decimale(126, 25)
```

$$126 \div 25 = 5.04$$

**Parameters**

```
egalite-decimale(
  a: int float,
  b: int float,
  s: str
) -> content
```

**a** `int` or `float`

dividend

**b** `int` or `float`

divisor

**s** `str`

sign:“e” for =, else approx

Default: `"e"`

### division

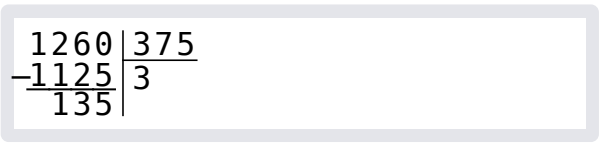
Divide decimal number `a` by `b`.

With `extradigits`, computes additional digits after the decimal point beyond those already present in `a` (when `cycle:false`). These extra digits are displayed in gray (`mode:"g"` by default), white (`mode:"0"`), as dots (`mode:"p"` or `"gp"`), or omitted (`mode:""`). Use `s:true/false` to show or hide the subtraction steps. You can highlight the repeating cycle in the quotient using `cycle:true` (underlined with `cycle-color`), display it in parentheses with `cycle:""`, or display it in color with

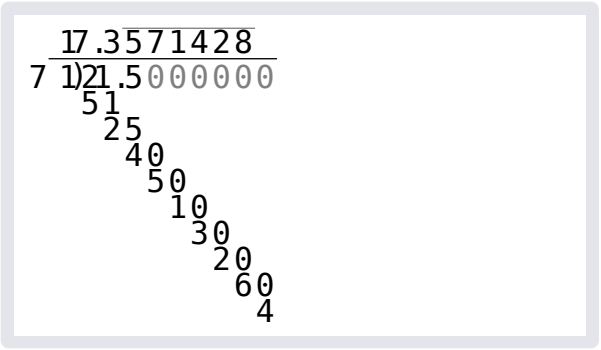
`cycle:red`. Fill-in-the-blank divisions are also supported with `hide-result`, `list`, and `solution` (using the `type-mask`, `border-color`, and `solution-color` parameters).

**Examples:**

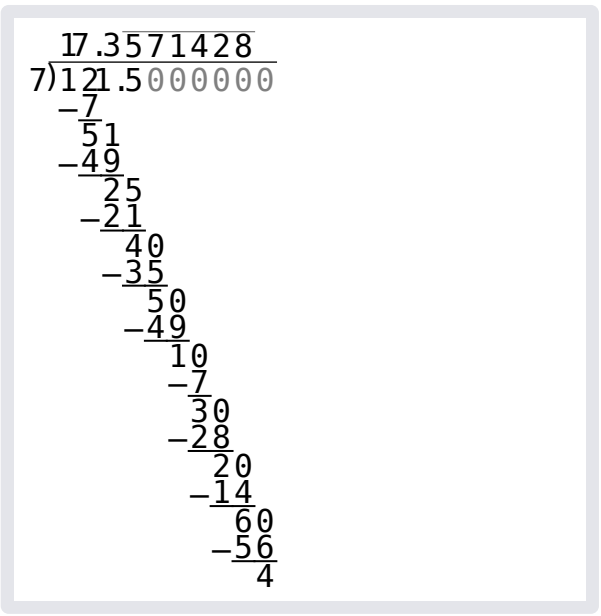
```
#division(12.6, 3.75)
```



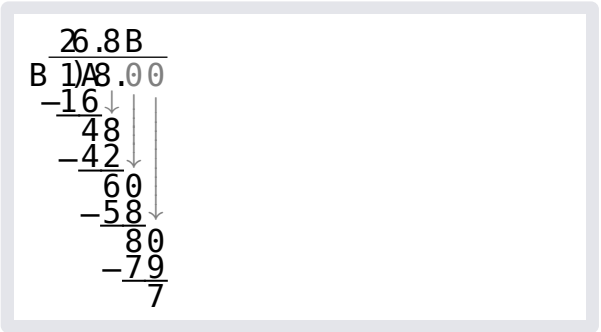

```
// you can get the cycle of repeating
digits automatically and hide the
subtractions:
#division(121.5, 7, cycle:true, s:
false, fr: false)
```




```
#division(121.5, 7,
cycle:true,fr:false,dx:.5em,dy:-.9em)
```




```
// in base 16 with a floating result
and arrows to show when digits are
pulled down:
#division(base: 16, extradigits: 2,
"1A8", "B", arrows: true, fr: false)
```



### Parameters

```
division(
  a: int float,
  b: int float,
  base: int,
  show-base: boolean,
  mode: str,
  extradigits: int,
  s: boolean,
  cycle: boolean str color,
  cycle-color: color,
  fr: boolean,
  size: length relative length,
  dx: length relative length,
  dy: length relative length,
  hide-result: boolean,
  list: array,
  solution: boolean,
  type-mask: str,
  border-color: color,
  solution-color: color,
  space: length relative length fraction
) -> content
```

**a** `int` or `float`

dividend

**b** `int` or `float`

divisor

**base** `int`

Base (2 to 36)

Default: `10`

**show-base** `boolean`

to show in which base the numbers and operation are

Default: `false`

**mode** `str`

Display mode for zeros added to the dividend: “g” (gray), “0” (black), “p” (small dot), “gp” (large dot), “” (empty)

Default: `"g"`

**extradigits** `int`

Number of additional quotient digits to compute

Default: `0`

**s** `boolean`

Show or hide the subtraction steps

Default: `true`

**cycle** `boolean` or `str` or `color`

Detect and display the repeating cycle automatically

Default: `false`

**cycle-color** `color`

Color of the cycle or underline

Default: `luma(50%)`

**fr** `boolean`

If fr:true, the decimal separator is “,” otherwise “.”

Default: `true`

**size** `length` or `relative length`

Column width

Default: `.7em`

**dx** `length` or `relative length`

dx for the ) in mode fr:false

Default: `.6em`

**dy** `length` or `relative length`

dy for the ) in mode fr:false

Default: `-.82em`

**hide-result** `boolean`

Hide the working steps and quotient (exercise mode)

Default: `false`

**list** `array`

List of cell indices to hide (dividend, divisor, steps, quotient)

Default: `()`

**solution** `boolean`

Show or hide the solution inside the masks

Default: `false`

**type-mask** `str`

Mask type (“rect” or “line”)

Default: `"rect"`

**border-color** `color`

Color of the mask border

Default: `blue.mix(gray)`

**solution-color** `color`

Color of the solution text

Default: `red`

**space** `length` or `relative length` or `fraction`

Spacing when several calculations are on the same line

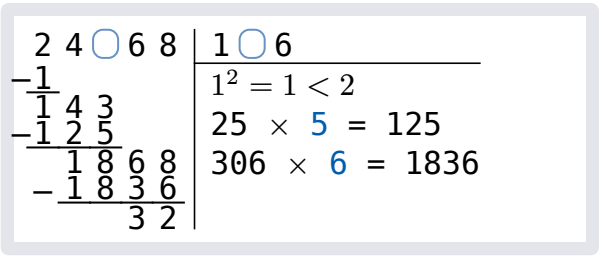
Default: `1fr`

**racine**

Compute the square root of `a`, with `extradigits` additional decimal places, optionally showing digit pairs using `groups` in `groups-color`, displaying the construction step by step with `step`, hiding subtraction steps with `s`, and hiding the result or a `list` of digits.

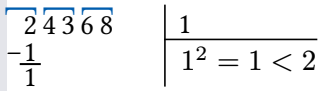
**Examples:**

```
#racine(24368,size: 1em,list: (2,27))
```

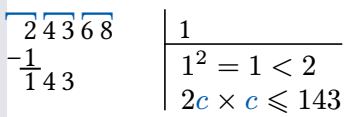



```
#grid(columns: 3*(1fr,), row-gutter: 1em,
 ..for i in range(1,11) {([
   #### Étape #i
   #racine(24368, extradigits: 1,groups: true, step: i, fr: false)
 ],)+if i == 1 {([],[],)}}
 )
```

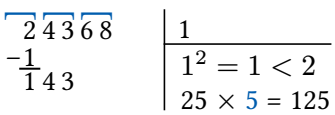
### Étape 1



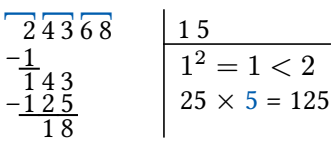

### Étape 2



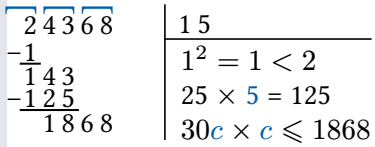

### Étape 3



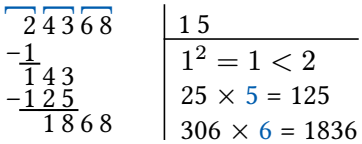

### Étape 4



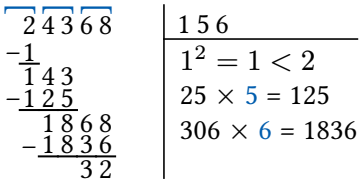

### Étape 5



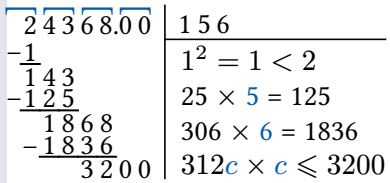

### Étape 6



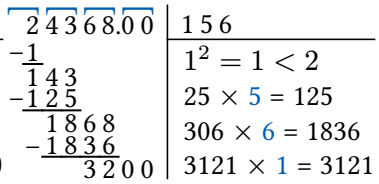

### Étape 7



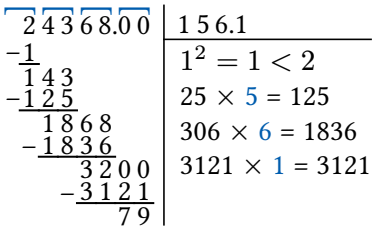

### Étape 8




### Étape 9




### Étape 10



### Parameters

```
racine(
  a: int float,
  extradigits: int,
  groups: boolean,
  groups-color: color,
  step: none int,
  step-color: color,
  s: boolean,
  fr: boolean,
  size: length relative length,
  hide-result: boolean,
  list: array,
  solution: boolean,
  type: str,
  border-color: color,
  solution-color: color,
  space: length relative length fraction,
  ..args: arguments
) -> content
```

**a** `int` or `float`

number

**extradigits** `int`

Number of additional “00” digit pairs to compute after the decimal point

Default: `0`

**groups** `boolean`

Show brackets above each pair of digits

Default: `false`

**groups-color** `color`

Color of the brackets

Default: `blue.darken(20%)`

**step** `none` or `int`

Step to display in the step-by-step construction

Default: `none`

**step-color** `color`

Color of the step indicator

Default: `blue.darken(20%)`

**s** `boolean`

Show or hide the subtraction steps below the dividend

Default: `true`

**fr** `boolean`

If fr:true, the decimal separator is “,” otherwise “.”

Default: `true`

**size** `length` or `relative length`

Column width

Default: `0.65em`

**hide-result** `boolean`

Hide the construction steps, operations, and result (exercise mode)

Default: `false`

**list** `array`

List of cell indices to hide (fill-in-the-blank exercise)

Default: `()`

**solution** `boolean`

Show or hide the solution inside the masks

Default: `false`

**type** `str`

Mask type (“rect” or “line”)

Default: `"rect"`

**border-color** `color`

Color of the mask border

Default: `blue.mix(gray)`

**solution-color** `color`

Color of the solution text

Default: `red`

**space** `length` or `relative length` or `fraction`

Right spacing

Default: `1fr`

**..args** `arguments`

Other arguments to the square root, ex:[...]

### add-en-ligne

Compute the sum of all the supplied numbers inline.

```
$#add-en-ligne(198.7,120,35)$
```

$198.7 + 120 + 35 = 353.7$

#### Parameters

```
add-en-ligne(..ops: arguments) -> content
```

**..ops** `arguments`

list of numbers

### sous-en-ligne

Compute the difference of all the supplied numbers inline.

```
$#sous-en-ligne(198.7,107.3,-35)$
```

$198.7 - 107.3 - (-35) = 126.4$

#### Parameters

```
sous-en-ligne(
  ..ops: arguments,
  digits: auto int
) -> content
```

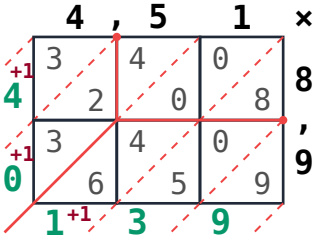
**..ops** `arguments`

list of numbers

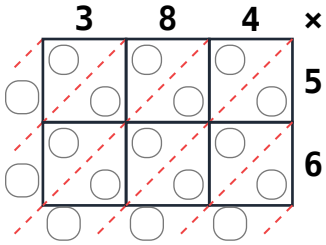
### grid-multiplication

Displays lattice multiplication (a grid with diagonals). This function generates a grid in which each cell is divided by a diagonal (tens in the upper-left triangle, units in the lower-right triangle). The sums are calculated along the diagonals.

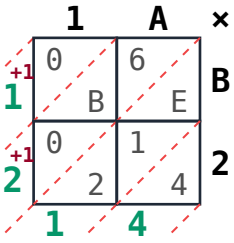
```
// Basic usage:
#grid-multiplication(4.51, 8.9,)
```




```
// Fill-in-the-blank grid
#grid-multiplication(384, 56, hide-
result: "all", type-mask: "box",)
```




```
// Base-16 (hexadecimal) calculation
#grid-multiplication("1A", "B2", base:
16)
```



### Parameters

```
grid-multiplication(
  a: int float decimal str,
  b: int float decimal str,
  cell-size: length,
  grid-color: color,
  comma-lines-color: color,
  diag-color: color,
  result-color: color,
  decimal-part-color: none color,
  numbers-color: color,
  prod-color: color,
  tens-color: none color,
  units-color: none color,
  solution-color: color,
  base: int,
  show-base: bool,
  hide-result: bool str,
  type-mask: str content function,
  list: array none,
  solution: bool,
  fr: bool
) -> content
```

**a** `int` or `float` or `decimal` or `str`

First factor (displayed at the top)

**b** `int` or `float` or `decimal` or `str`

Second factor (displayed on the right)

**cell-size** `length`

Size of the grid cells

Default: `1.1cm`

**grid-color** `color`

Grid color

Default: `rgb("1f2937")`

**comma-lines-color** `color`

Color of the lines indicating the decimal point position

Default: `rgb("ef4444")`

**diag-color** `color`

Diagonal color

Default: `rgb("ef4444")`

**result-color** `color`

Result color

Default: `rgb("059669")`

**decimal-part-color** `none` or `color`

Color of the decimal part of the result

Default: `none`

**numbers-color** `color`

Factor color

Default: `black`

**prod-color** `color`

Partial products color

Default: `luma(80)`

**tens-color** `none` or `color`

Tens color

Default: `none`

**units-color** `none` or `color`

Units color

Default: `none`

**solution-color** `color`

Color used to display the solution

Default: `rgb("2563eb")`

**base** `int`

Number base (default: 10)

Default: `10`

**show-base** `bool`

Show the number base

Default: `false`

**hide-result** `bool` or `str`

Masking options: false (show everything), true or “result” (hide the result), “grid”, “grid-only” or “all” (empty grid with factors only), “calculations” (hide calculations but show the result)

Default: `false`

**type-mask** `str` or `content` or `function`

Style used for hidden values (“blank”, “box”, “question”, “dots”, content or function)

Default: `"blank"`

**list** `array` or `none`

List of target elements or specific coordinates (column, row) to hide

Default: `none`

**solution** `bool`

Enable correction mode: display hidden values in solution-color

Default: `false`

**fr** `bool`

Use a comma as the decimal separator

Default: `true`

**binary-multiplication**

Displays Ethiopian multiplication in table form. This method relies on the binary decomposition of the first factor: successive divisions by 2 (left column) and multiplications by 2 (right column). Rows in which the left-hand number is even (a 0 in binary notation) are crossed out, and the remaining numbers in the right column are added together.

```
#binary-multiplication(23, 14, show-equality: false,sums: true, sum-symbol: "Result:")
```

| 23 | 14 |
|---|---|
| ÷ 2 | × 2 |
| 23 | 14 |
| 11 | 28 |
| 5 | 56 |
| ~~2~~ | ~~112~~ |
| 1 | 224 |
| Result: | 322 |

```
// With decimal numbers and explanations
#binary-multiplication("2.7", 1.3,numbers: true,rule: true,sums: true, )
```

| 2,7 | 1,3 | $n$ |
|---|---|---|
| ÷ 2 | × 2 | |
| 27 | 13 | $1 \times 13$ |
| 13 | 26 | $2 \times 13$ |
| ~~6~~ | ~~52~~ | ~~$4 \times 13$~~ |
| 3 | 104 | $8 \times 13$ |
| 1 | 208 | $16 \times 13$ |
| $\sum$ | **351** | $27 \times 13$ |

$$27 \times 13 = (1 + 2 + 8 + 16) \times 13 = 13 + 26 + 104 + 208 = \mathbf{351}$$

$$2{,}7 \times 1{,}3 = \mathbf{3{,}51}$$

## Parameters

```
binary-multiplication(
  a: int float decimal str,
  b: int float decimal str,
  numbers: bool,
  rule: bool,
  sums: bool,
  sum-symbol: content,
  biffer-pairs: bool,
  show-equality: bool,
  fill-even: color,
  fill-odd: color,
  stroke: stroke,
  fr: bool
) -> content
```

**a** `int` or `float` or `decimal` or `str`

First factor

**b** `int` or `float` or `decimal` or `str`

Second factor

**numbers** `bool`

Explain the decomposition into partial products

Default: `false`

**rule** `bool`

Display ÷ 2 and × 2 to explain the calculations

Default: `true`

**sums** `bool`

Display a final row containing the sums

Default: `false`

**sum-symbol** `content`

Summation symbol or label

Default: `$sum$`

**biffer-pairs** `bool`

Cross out rows containing even numbers

Default: `true`

**show-equality** `bool`

Display an equality below the table

Default: `true`

**fill-even** `color`

Background color for rows containing even numbers

Default: `rgb("ffebee")`

**fill-odd** `color`

Background color for rows containing odd numbers

Default: `rgb("e8f5e9")`

**stroke** `stroke`

Table borders

Default: `0.5pt + luma(180)`

**fr** `bool`

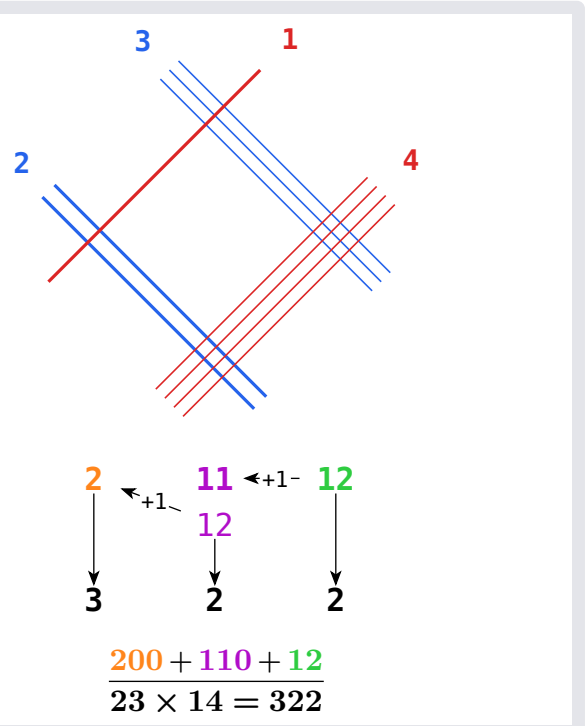
Use a comma as the decimal separator

Default: `true`

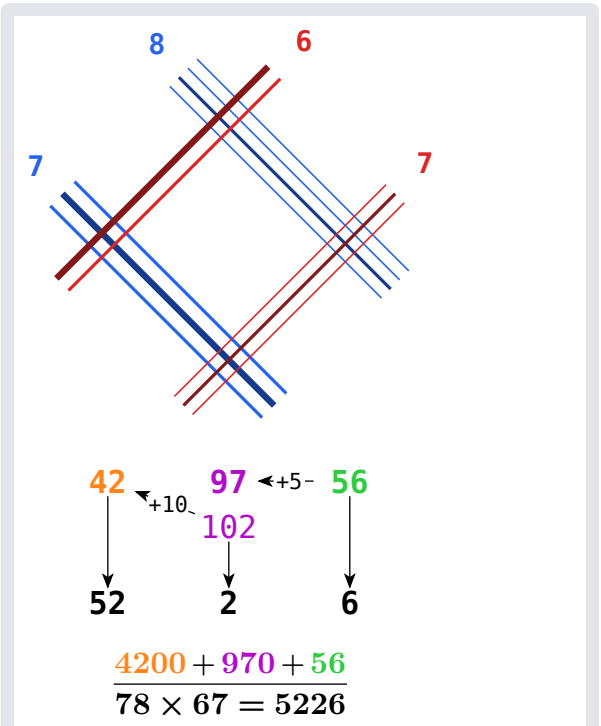
### lines-multiplication

Japanese multiplication (line multiplication method). Each digit is represented by a bundle of parallel lines. The intersections indicate the number of units, tens and hundreds.

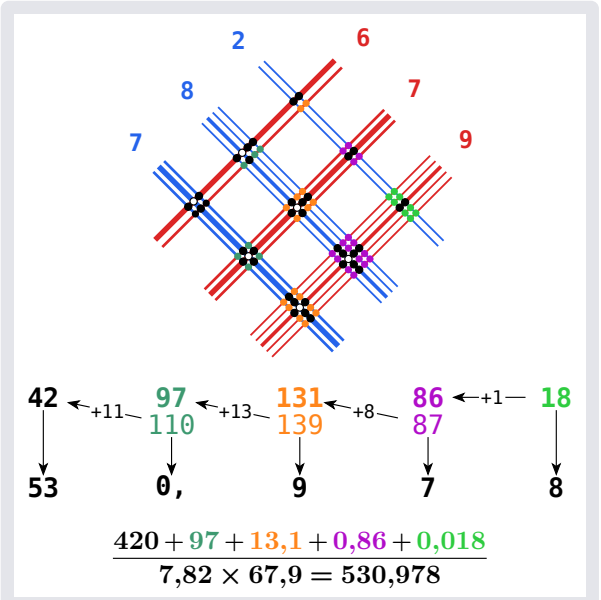
```
// Basic usage
#lines-multiplication(23, 14)
```



```
// "Improved" option (every group of 5 lines is replaced by a single darker, thicker line)
#lines-multiplication(78, 67, improved: true,)
```




```
// With visible intersections — a black dot represents 5 and a white dot represents 25 when `improved:true`:
#lines-multiplication(7.82, 67.9, improved: true, points: true, print: true, digits-spread-factor: .5, lines-spread-factor: .7, counts-y-spread-factor: .7,)
```



### Parameters

```
lines-multiplication(
  a: int float decimal str,
  b: int float decimal str,
  color-a: color,
  color-b: color,
  color-zero: color,
  draw-scale: int float,
  fr: bool,
  points-color: array color,
  color-blocs: color none,
  blocs-scale: float int,
  lines-thickness: length,
  min-thickness: length,
  points: bool,
  digits-spread-factor: float,
  lines-spread-factor: float,
  counts-x-spread-factor: float,
  counts-y-spread-factor: float,
  improved: bool,
  color-5a: color none,
  color-5b: color none,
  color-5: color,
  color-25: color,
  blocs: bool str,
  counts: bool,
  result: bool,
  only-drawing: bool,
  only-calculation: bool,
  equal: bool,
  carries: bool,
  sum: bool,
  print: bool
) -> content
```

**a** `int` or `float` or `decimal` or `str`

First factor

**b** `int` or `float` or `decimal` or `str`

Second factor

**color-a** `color`

Color of the lines representing a

Default: `rgb("2563eb")`

**color-b** `color`

Color of the lines representing b

Default: `rgb("dc2626")`

**color-zero** `color`

Color of dashed lines representing zeros

Default: `gray`

**draw-scale** `int` or `float`

Drawing scale factor

Default: `.8`

**fr** `bool`

Use a comma or a period as the decimal separator

Default: `true`

**points-color** `array` or `color`

Colors of intersection points representing 1, from units to higher place values

Default: `(green, purple, orange, olive, black, red, blue,)`

**color-blocs** `color` or `none`

Color of the blocks surrounding intersection points

Default: `none`

**blocs-scale** `float` or `int`

Block scale factor

Default: `0.7`

**lines-thickness** `length`

Thickness of the unit lines

Default: `0.6pt`

**min-thickness** `length`

Minimum line thickness (for decimal digits)

Default: `0.4pt`

**points** `bool`

Display intersection points (e.g. in “Baguettes” mode)

Default: `false`

**digits-spread-factor** `float`

Spacing factor for digits

Default: `1`

**lines-spread-factor** `float`

Spacing factor between the lines representing a digit

Default: `1`

**counts-x-spread-factor** `float`

Horizontal spacing factor for result values in each region

Default: `1`

**counts-y-spread-factor** `float`

Vertical spacing factor for result values in each region

Default: `1`

**improved** `bool`

Replace every group of 5 lines with a single line of the opposite color (for digits ≥ 5)

Default: `false`

**color-5a** `color` or `none`

Color of lines representing 5 in the first factor

Default: `none`

**color-5b** `color` or `none`

Color of lines representing 5 in the second factor

Default: `none`

**color-5** `color`

Color of intersection points representing 5

Default: `black`

**color-25** `color`

Color of intersection points representing 25 (two condensed lines crossing)

Default: `white`

**blocs** `bool` or `str`

Place-value blocks: true, false, “rect” or “circ”

Default: `false`

**counts** `bool`

Display sums, carries and digits below the drawing

Default: `true`

**result** `bool`

Display the result after applying carries

Default: `true`

**only-drawing** `bool`

Display only the drawing

Default: `false`

**only-calculation** `bool`

Display only the calculation explanation

Default: `false`

**equal** `bool`

Display the resulting equality

Default: `true`

**carries** `bool`

Display carries

Default: `true`

**sum** `bool`

Display the sum

Default: `true`

**print** `bool`

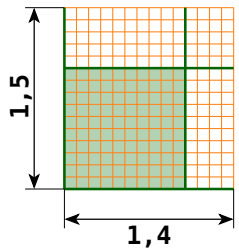
Print mode: use uniform line thickness and double the thickness of lines representing 5

Default: `false`

### area-multiplication

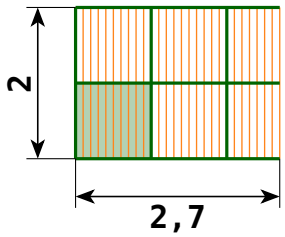
Visualizes the multiplication of decimal numbers using a rectangle with dimensions a × b. The "1 × 1" unit square is colored, thin lines mark tenths (if the corresponding factor is decimal), thick lines mark whole units, and dimension labels are shown.

```
#area-multiplication("1.5", "1.4")
```




$1{,}5 \times 1{,}4 = 15 \times 0{,}1 \times 14 \times 0{,}1 = 210 \times 0{,}01 = 2{,}1$

```
#area-multiplication(2, "2.7", unit:
1cm)
```




$2 \times 2{,}7 = 2 \times 27 \times 0{,}1 = 54 \times 0{,}1 = 5{,}4$

### Parameters

```
area-multiplication(
  a: int float str,
  b: int float str,
  unit: length,
  tenths-color: color,
  ones-color: color,
  unit-color: color auto,
  tenths-a: bool auto,
  tenths-b: bool auto,
  fr: bool,
  tenths-thickness: length,
  units-thickness: length,
  show-result
) -> content
```

**a** `int` or `float` or `str`

First factor

**b** `int` or `float` or `str`

Second factor

**unit** `length`

Side length of one unit

Default: `2cm`

**tenths-color** `color`

Color of the tenths lines

Default: `orange`

**ones-color** `color`

Color of the units lines

Default: `rgb("006400")`

**unit-color** `color` or `auto`

Fill color of the “1 × 1” square (auto: a lighter version of ones-color)

Default: `auto`

**tenths-a** `bool` or `auto`

Tenths lines parallel to a: auto (drawn if a is decimal) | true | false

Default: `auto`

**tenths-b** `bool` or `auto`

Tenths lines parallel to b: auto (drawn if b is decimal) | true | false

Default: `auto`

**fr** `bool`

Use a comma or a period as the decimal separator

Default: `true`

**tenths-thickness** `length`

Thickness of the tenths lines

Default: `0.5pt`

**units-thickness** `length`

Thickness of the units lines

Default: `1.2pt`

## Additions and substractions of durations


- addition-durees()
- prepa-duree()
- soustraction-durees()


### addition-durees

Sets up the addition of two or more durations.

You need to change units: ([j], [h], [min], [s]) with units: ([d], [h], [min], [s]).

Each duration is an array in one of the following forms:

- `(days, hours, minutes, seconds)`.
- `(hours, minutes, seconds)`.
- `(minutes, seconds)`.

**Examples:**

```
#addition-durees(
  (1, 2, 45, 50),
  (0, 1, 20, 30),
)
```

| | j | h | min | s |
|---|---|---|---|---|
| | | 1 | 1 | |
| | 1 | 02 | 45 | 50 |
| + | 0 | 01 | 20 | 30 |
| | 1 | 04 | 06 | 20 |

```
#addition-durees(
  (
    (1, 2, 45, 50),
    (0, 1, 20, 30),
    (2, 23, 55, 45),
  ),
  hide-result: true,
)
```

| | j | h | min | s |
|---|---|---|---|---|
| | 1 | 2 | 2 | |
| | 1 | 02 | 45 | 50 |
| + | 0 | 01 | 20 | 30 |
| + | 2 | 23 | 55 | 45 |
| | | | | |

### Parameters

```
addition-durees(
  show-carry: boolean,
  carries-color: color,
  carry-size: length relative length,
  size: length relative length,
  convertion: boolean,
  hide-empty: boolean,
  hide-result: boolean,
  border-color: color,
  solution-color: color,
  type-mask: str,
  list: array,
  solution: boolean,
  sign: boolean content,
  text-size: length relative length,
  show-units: boolean,
  units: array,
  units-size: length relative length,
  units-color: color,
  zero-pad: boolean,
  align: boolean,
  verticals: boolean,
  ..args: array arguments
) -> content
```

**show-carry** `boolean`

Show carries.

Default: `true`

**carries-color** `color`

Color of the carries.

Default: `red`

**carry-size** `length` or `relative length`

Size of the carries and the + and = symbols, max. 1em.

Default: `.8em`

**size** `length` or `relative length`

Width of the columns corresponding to the units.

Default: `2.5em`

**convertion** `boolean`

Convert the result instead of displaying carries.

Default: `false`

**hide-empty** `boolean`

Hide columns containing only zeros.

Default: `true`

**hide-result** `boolean`

Hide the result.

Default: `false`

**border-color** `color`

Border color of hidden values.

Default: `blue.mix(gray)`

**solution-color** `color`

Color of values in the solution.

Default: `red`

**type-mask** `str`

Mask type: “rect” or “line”.

Default: `"rect"`

**list** `array`

Indices of the cells to hide.

Default: `()`

**solution** `boolean`

Show the solution for hidden cells.

Default: `false`

**sign** `boolean` or `content`

Sign placed before the result.

Default: `false`

**text-size** `length` or `relative length`

Text size.

Default: `1em`

**show-units** `boolean`

Show unit names.

Default: `true`

**units** `array`

Names of the four units.

Default: `([d], [h], [min], [s])`

**units-size** `length` or `relative length`

Unit size, max. 1em.

Default: `.75em`

**units-color** `color`

Color of the units.

Default: `gray`

**zero-pad** `boolean`

Display hours, minutes, and seconds using two digits.

Default: `true`

**align** `boolean`

Align the digits of the units.

Default: `true`

**verticals** `boolean`

Vertical dotted lines to separate the columns.

Default: `false`

**..args** `array` or `arguments`

List of durations

**prepa-duree**

Converts decimal or fractional durations into integer durations

**Parameters**

```
prepa-duree(..args: array arguments) -> array
```

**..args** `array` or `arguments`

List of 2 to 4 numbers (d, h, min, s) to (min, s)

**soustraction-durees**

Sets up the subtraction of two durations.

You need to change units: ([j], [h], [min], [s]) with units: ([d], [h], [min], [s]).

Each duration is an array in one of the following forms:

- `(days, hours, minutes, seconds)`;
- `(hours, minutes, seconds)`;
- `(minutes, seconds)`.

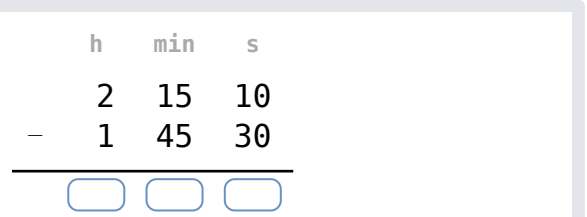
The first duration must be greater than or equal to the second.

**Examples:**

```
#soustraction-durees(
  (2, 4, 15, 10),
  (1, 6, 45, 30),
)
```

| | j | h | min | s |
|---|---|---|---|---|
| | | +24 | +60 | +60 |
| | 2 | 04 | 15 | 10 |
| − | 1 | 06 | 45 | 30 |
| | +1 | +1 | +1 | |
| | 0 | 21 | 29 | 40 |

```
#soustraction-durees(
  (2, 15, 10),
  (1, 45, 30),
  hide-result: true,
)
```




**Parameters**

```
soustraction-durees(
  show-borrow: boolean,
  carries-color: color,
  carry-size: length relative length,
  size: length relative length,
  convertion: boolean,
  hide-empty: boolean,
  hide-result: boolean,
  border-color: color,
  solution-color: color,
  type-mask: str,
  list: array,
  solution: boolean,
  sign: boolean content,
  text-size: length relative length,
  show-units: boolean,
  units: array,
  units-size: length relative length,
  units-color: color,
  zero-pad: boolean,
  align: boolean,
  verticals: boolean,
  ..args: array arguments
) -> content
```

**show-borrow** `boolean`

Show borrows and carries.

Default: `true`

**carries-color** `color`

Color of the borrows and carries.

Default: `red`

**carry-size** `length` or `relative length`

Size of the carries and the + and = symbols, max. 1em.

Default: `.7em`

**size** `length` or `relative length`

Width of the columns.

Default: `2.5em`

**convertion** `boolean`

Convert the minuend instead of displaying carries.

Default: `false`

**hide-empty** `boolean`

Hide columns containing only zeros.

Default: `true`

**hide-result** `boolean`

Hide the result.

Default: `false`

**border-color** `color`

Border color of hidden values.

Default: `blue.mix(gray)`

**solution-color** `color`

Color of values in the solution.

Default: `red`

**type-mask** `str`

Mask type: “rect” or “line”.

Default: `"rect"`

**list** `array`

Indices of the cells to hide.

Default: `()`

**solution** `boolean`

Show the solution.

Default: `false`

**sign** `boolean` or `content`

Sign placed before the result.

Default: `false`

**text-size** `length` or `relative length`

Text size.

Default: `1em`

**show-units** `boolean`

Show unit names.

Default: `true`

**units** `array`

Names of the four units.

Default: `([d], [h], [min], [s])`

**units-size** `length` or `relative length`

Unit size, max. 1em.

Default: `.75em`

**units-color** `color`

Color of the units.

Default: `gray`

**zero-pad** `boolean`

Display hours, minutes, and seconds using two digits.

Default: `true`

**align** `boolean`

Align the digits of the units.

Default: `true`

**verticals** `boolean`

Vertical dotted lines to separate the columns.

Default: `false`

**..args** `array` or `arguments`

List of durations

## Step-by-step calculations and highlighting

- detail()
- etapes-calcul()

### detail

Function for writing calculation steps manually.

```
#detail("A=15/21=(5*r3)/
(7*r3)=5/7",add-
rounded:true,c:"circle")
```

$$\mathrm{A} = \frac{15}{21} = \frac{5 \times \textcircled{3}}{7 \times \textcircled{3}} = \frac{5}{7} \approx 0{,}71$$

```
#detail("B=(15s2)/(21s18)=(5*r3 s2)/
(7*r3 s(9*2))=(5 ps2)/(21 bs2)
=5/21",c:"color",vertical:true, add-
rounded: true)
```

$$\begin{aligned}
\mathrm{B} &= \frac{15\sqrt{2}}{21\sqrt{18}} \\
&= \frac{5 \times 3\sqrt{2}}{7 \times 3\sqrt{9 \times 2}} \\
&= \frac{5\sqrt{2}}{21\sqrt{2}} \\
&= \frac{5}{21} \\
&\approx 0{,}24
\end{aligned}$$

```
#detail("cal(A)=pi * r^2 = pi*5^2=25
pi",add-rounded: true,digits: 4)
```

$$\mathcal{A} = \pi \times r^2 = \pi \times 5^2 = 25\pi \approx 25 \times 3{,}1416 \approx 78{,}54 \approx 78{,}54$$

### Parameters

```
detail(
  expr-str: str,
  vertical: boolean,
  add-rounded: boolean,
  digits: int,
  appro: int,
  todo: boolean,
  space: length relative length fraction,
  a: color,
  b: color,
  f: color,
  g: color,
  r: color,
  l: color,
  m: color,
  n: color,
  o: color,
  t: color,
  p: color,
  y: color,
  c: str
) -> content
```

**expr-str** `str`

The detailed calculation expression

**vertical** `boolean`

Vertical layout

Default: `false`

**add-rounded** `boolean`

Append a rounded value at the end

Default: `false`

**digits** `int`

Number of decimal places for the rounded value

Default: `2`

**appro** `int`

Step from which “=” is replaced by “≈” when appropriate

Default: `0`

**todo** `boolean`

If `todo:true`, only the initial expression is displayed

Default: `false`

**space** `length` or `relative length` or `fraction`

Spacing before the next calculation

Default: `1fr`

**a** `color`

`a`: aqua; can be changed from Typst’s default color if needed

Default: `aqua`

**b** `color`

`b`: blue; can be changed from Typst’s default color if needed

Default: `blue`

**f** `color`

`f`: fuchsia; can be changed from Typst's default color if needed

Default: `fuchsia`

**g** `color`

`g`: green; can be changed from Typst's default color if needed

Default: `green`

**r** `color`

`r`: red; can be changed from Typst's default color if needed

Default: `red`

**l** `color`

`l`: gray; can be changed from Typst's default color if needed

Default: `gray`

**m** `color`

`m`: maroon; can be changed from Typst's default color if needed

Default: `maroon`

**n** `color`

`n`: rgb(8,111,189) (navy); can be changed if needed

Default: `rgb("#086fbd")`

**o** `color`

`o`: orange; can be changed from Typst's default color if needed

Default: `orange`

**t** `color`

`t`: rgb(0,128,128) (teal); can be changed if needed

Default: `rgb(0,128,128)`

**p** `color`

p: purple; can be changed from Typst's default color if needed

Default: `purple`

**y** `color`

y: yellow; can be changed from Typst's default color if needed

Default: `yellow`

**c** `str`

Display mode: "cancel", "circle", or "color"

Default: `"color"`

**etapes-calcul**

Show the step-by-step evaluation of `expr-str` (unless `todo:true`), with the option to perform independent operations of the same precedence simultaneously using `concomitant`, display the work vertically or horizontally, highlight the current step (`highlight`), and use fraction mode to keep computations exact by forcing calculations to continue with fractions.

```
#etapes-calcul("[ 6 + 5 * (23 + 8 /
2) ] * 8", fraction: false, vertical:
true,n:0)
```

$$\begin{aligned}&\left[6+5\times\left(23+\frac{8}{2}\right)\right]\times 8\\ &=[6+5\times(23+4)]\times 8\\ &=[6+5\times 27]\times 8\\ &=[6+135]\times 8\\ &=141\times 8\\ &=1128\end{aligned}$$

```
#etapes-calcul("(200 - 45 * 2) / ((4 *
7 - 3 : 6) * 2)",concomitant: true)
```

$$\frac{200-45\times 2}{(4\times 7-3\div 6)\times 2}=\frac{200-90}{(28-0{,}5)\times 2}=\frac{110}{27{,}5\times 2}=\frac{110}{55}=2$$

```
#etapes-calcul("2 * pi * 3", fraction:
false, name: "B",n:3)
```

$$B=2\times\pi\times 3=2\pi\times 3=6\pi\approx 18{,}85$$

```
#etapes-calcul("ln(exp(1)) + 3",
fraction: true, name: "E")
```

$$E=\ln(\exp(1))+3=1+3=4$$

```
#etapes-calcul("sqrt(8)*sqrt(27)",
fraction: true)
```

$$\sqrt{8} \times \sqrt{27} = 2\sqrt{2} \times \sqrt{27} = 2\sqrt{2} \times 3\sqrt{3} = 6\sqrt{6}$$

**Parameters**

```
etapes-calcul(
  expr-str: str,
  mode: str,
  concomitant: boolean,
  vertical: boolean,
  digits: int,
  n: int,
  name: str content,
  todo: boolean,
  space: length relative length fraction,
  fraction: boolean,
  highlight: color boolean none,
  skip: array
) -> content
```

**expr-str** `str`

expression to calculate

**mode** `str`

“fr” or “en” mode for the decimal separator

Default: `"fr"`

**concomitant** `boolean`

Set to true to perform independent operations of the same precedence simultaneously

Default: `false`

**vertical** `boolean`

Display the calculation vertically

Default: `false`

**digits** `int`

Number of decimal places for calculations involving π, etc.

Default: `2`

**n** `int`

Number of steps (counted from the end) at which “=” is replaced by “≈” when appropriate

Default: `0`

**name** `str` or `content`

Label to prepend to the expression

Default: `""`

**todo** `boolean`

Exercise mode: only the initial expression followed by “=” is displayed

Default: `false`

**space** `length` or `relative length` or `fraction`

Horizontal spacing between two `etapes-calcul` outputs

Default: `1fr`

**fraction** `boolean`

In fraction mode, computations remain exact, either as fractions or simplified forms such as $5\pi$

Default: `false`

**highlight** `color` or `boolean` or `none`

Either a color used to highlight the current operation, `false` for bold, or `none` for no highlighting

Default: `rgb("#1e88e5")`

**skip** `array`

List of steps to skip if needed

Default: `()`